

LABORATORIO DI INFORMATICA

Modulo M1

Markdown, documentazione e ambiente Jupyter

Esercizi

Istituto Tecnico Tecnologico "G. Marconi" — Rovereto

Indirizzo Informatica e Telecomunicazioni, articolazione Informatica

Classe 4Bi — Anno scolastico 2026-2027 · Docente di laboratorio: Andrea Trentini

Modulo M1 — Esercizi su Markdown, documentazione e ambiente Jupyter

Gli esercizi su Markdown si svolgono nel repository personale del corso, dentro la cartella `M1_markdown/`; gli esercizi sul notebook si svolgono sul Jupyter server d'istituto, nella cartella personale, e i file prodotti vanno poi copiati nel repository dentro la cartella `M1_jupyter/`. Ogni esercizio produce almeno un file che va aggiunto al repository con un `commit`. Il messaggio rispetta la convenzione fissata nel modulo M0, cioè una prima riga di massimo cinquanta caratteri al presente indicativo e senza punto finale, e richiama fra parentesi il numero dell'esercizio: per esempio `Aggiunge la pagina delle risorse (M1 es. 3)`.

Prima di ogni consegna va controllata l'anteprima di Visual Studio Code (`Ctrl+K` seguito da `V`) e, per gli esercizi che lo richiedono espressamente, anche il risultato reso su GitHub. Per gli esercizi che producono un notebook la verifica obbligatoria è `Kernel > Restart Kernel and Run All Cells`.

Tutti i blocchi di codice devono riportare l'indicazione del linguaggio. Non è ammesso l'uso di HTML dentro i file Markdown e dentro le celle di testo dei notebook, salvo dove l'esercizio lo chieda esplicitamente.

Livello base

Esercizio 1 — Scheda personale di postazione

Creare il file `M1_markdown/postazione.md` che descrive la propria postazione di laboratorio. Il documento deve avere un titolo di primo livello, tre sezioni di secondo livello dedicate rispettivamente all'hardware, al software installato e alle credenziali di accesso ai servizi del corso (senza riportare alcuna password), e almeno una sottosezione di terzo livello. Ogni sezione contiene almeno due paragrafi separati da riga vuota. Va usato il corsivo almeno una volta e il grassetto al massimo tre volte in tutto il documento.

Dati di esempio da adattare alla propria postazione: PC Windows 11, processore Intel i7, 16 GB di memoria, Visual Studio Code versione 1.9x, JDK 17, Git 2.4x, account GitHub personale, indirizzo del Jupyter server d'istituto.

Output atteso: nell'anteprima si vedono un titolo grande, tre intestazioni di sezione più piccole, una sottosezione ancora più piccola e paragrafi distinti fra loro. Nessun paragrafo deve risultare fuso con il successivo.

Tempo stimato: 20 minuti

Esercizio 2 — Istruzioni di compilazione con elenchi, comandi e citazioni

Creare il file `M1_markdown/compilare.md` con le istruzioni per compilare ed eseguire un programma Java a partire da un repository appena clonato. Le istruzioni vanno scritte come elenco numerato di almeno cinque passi, di cui almeno uno con un sotto-elenco annidato, e devono contenere almeno due blocchi di codice `bash` con i comandi esatti da digitare. Nel testo dei passi, ogni riferimento a un nome di file, a un'opzione o a una cartella va scritto in linea con le virgolette inverse. Al termine, riportare in una citazione il messaggio di errore che si ottiene se il JDK non è installato, e sotto la citazione la frase che spiega come rimediare.

Input di esempio: il progetto ha i sorgenti in `src/`, va compilato nella cartella `bin/` e la classe con il metodo `main` si chiama `MediaVoti`.

Output atteso: un elenco numerato 1, 2, 3, 4, 5 con un livello annidato correttamente rientrato, due riquadri di codice con colorazione sintattica, e un blocco citato reso con la barra verticale o il rientro tipico delle citazioni. Un blocco di codice privo dell'indicazione `bash` è considerato errore.

Tempo stimato: 30 minuti

Esercizio 3 — Pagina delle risorse del corso

Creare il file `M1_markdown/risorse.md` che raccoglie i collegamenti utili del corso. Devono comparire almeno tre collegamenti a indirizzi esterni con testo descrittivo, almeno un collegamento relativo a un altro file del proprio repository, e un'immagine con testo alternativo descrittivo. L'immagine va salvata nella cartella `M1_markdown/img/` e richiamata con un percorso relativo. Non è ammesso alcun indirizzo assoluto che punti al disco locale.

Input di esempio: il sito ufficiale di Markdown, la specifica CommonMark, la documentazione di Pandoc, il proprio file `README.md`, un'immagine dello schermo dell'anteprima di Visual Studio Code.

Output atteso: nell'anteprima i tre collegamenti sono cliccabili e mostrano il testo descrittivo, non l'indirizzo; l'immagine è visibile. Dopo il `push`, gli stessi elementi devono funzionare anche nella pagina resa da GitHub.

Tempo stimato: 25 minuti

Esercizio 4 — Le varianti di Markdown a confronto

Creare il file `M1_markdown/varianti.md` contenente una tabella che confronta Markdown originale, CommonMark e GitHub Flavored Markdown. Le righe corrispondono ai costrutti: blocchi di codice recintati, tabelle, caselle di spunta, testo barrato, collegamenti automatici, note a piè di pagina. Le colonne sono il costrutto e le tre varianti. La prima colonna va allineata a sinistra e le altre tre centrate.

Sotto la tabella, una sezione di esempi mostra all'opera le estensioni di GitHub Flavored Markdown: almeno sei caselle di spunta di cui due già spuntate, almeno un riferimento a una versione superata reso con il testo barrato, e almeno un indirizzo scritto per esteso che diventi cliccabile grazie al collegamento automatico. Chiude un paragrafo di almeno quattro righe che spiega in prosa quale variante conviene usare per le consegne del corso, perché, e che cosa succede a questo stesso file se viene aperto con uno strumento che implementa solo CommonMark.

Output atteso: una tabella a quattro colonne e sette righe, intestazione compresa, con l'allineamento richiesto. Su GitHub le caselle compaiono come quadratini, due dei quali spuntati, l'indirizzo è cliccabile e il testo barrato è attraversato da una linea. Il confronto va fatto anche nell'anteprima di Visual Studio Code, annotando nel paragrafo finale le eventuali differenze osservate.

Tempo stimato: 35 minuti

Esercizio 5 — Il primo notebook personale

Sul Jupyter server d'istituto, creare nella propria cartella un notebook chiamato `M1_esercizio5.ipynb` con il kernel `Python 3`. Il notebook deve contenere almeno tre celle di testo e almeno tre celle di codice, alternate. Le celle di testo usano la sintassi Markdown della prima parte del modulo: la prima contiene un titolo di primo livello con il proprio nome e la data, le altre due contengono almeno un elenco, una tabella e due riferimenti in linea fra virgolette inverse. Le celle di codice si limitano a un'operazione aritmetica il cui risultato compare come valore dell'ultima espressione, a un assegnamento di variabile e a una stampa con `print` che riusa la variabile assegnata nella cella precedente.

Input di esempio: le ore del proprio orario settimanale, il numero di studenti della classe, il numero di settimane del modulo.

Output atteso: eseguendo `Kernel > Restart Kernel and Run All Cells` il notebook arriva in fondo senza errori, i numeri fra parentesi quadre sono crescenti dall'alto verso il basso, e le celle di testo mostrano il testo reso e non il sorgente. Il file va scaricato con `Download` e copiato in `M1_jupyter/`.

Tempo stimato: 30 minuti

Livello intermedio

Esercizio 6 — README di un progetto Java del terzo anno

Scegliere un proprio progetto Java realizzato al terzo anno e scrivere il suo `README.md` completo, salvandolo in `M1_markdown/readme-progetto.md`. Il documento deve contenere, nell'ordine, una descrizione del programma in non più di tre frasi, una sezione sui requisiti con tabella,

una sezione di installazione con elenco numerato e blocchi di codice, una sezione d'uso con almeno un esempio completo di comando, formato del file di ingresso e output atteso in un blocco `text`, una sezione sulla struttura del progetto e una sezione finale con autore e licenza.

Input di esempio: il progetto legge un file CSV e produce un riepilogo a video. Se non si dispone di un progetto proprio, usare come riferimento il programma `MediaVoti` descritto nella dispensa.

Output atteso: un documento in cui il lettore che non conosce il progetto riesce, leggendo dall'alto in basso e senza saltare, a installare ed eseguire il programma. La prova è concreta: un compagno deve riuscirci senza fare domande.

Tempo stimato: 45 minuti

Esercizio 7 — Relazione di laboratorio con indice interno e note

Creare il file `M1_markdown/relazione.md`, una relazione su un'esercitazione di laboratorio già svolta, strutturata in obiettivo, materiale, procedimento, risultati e conclusioni. In cima al documento va un indice costruito a mano con collegamenti interni alle cinque sezioni. Il documento deve contenere almeno due note a piè di pagina, una tabella dei risultati con almeno quattro righe, e almeno un blocco di codice.

Input di esempio: i tempi di esecuzione, in millisecondi, di un algoritmo di ordinamento su array di dimensione 100, 1000, 10000 e 100000.

Output atteso: cliccando una voce dell'indice nell'anteprima si salta alla sezione corrispondente; le note compaiono numerate in fondo al documento con il collegamento di ritorno.

Tempo stimato: 45 minuti

Esercizio 8 — Guida ai recinti annidati

Creare il file `M1_markdown/recinti.md`, una breve guida che spiega a un compagno come si mostra un blocco di codice dentro un altro blocco di codice. La guida deve contenere almeno tre esempi crescenti: un blocco `java` normale, lo stesso blocco mostrato come sorgente dentro un recinto più esterno, e un terzo livello di annidamento. Ogni esempio è preceduto da un paragrafo che spiega che cosa il lettore vedrà.

Attenzione: questo esercizio si corregge quasi solo guardando l'anteprima. Un recinto sbagliato rende tutto il resto del documento come codice.

Output atteso: nell'anteprima i tre esempi si distinguono chiaramente, l'ultimo paragrafo del documento è reso come testo normale e non come codice, e il file convertito in HTML con Pandoc presenta la stessa struttura dell'anteprima.

Tempo stimato: 35 minuti

Esercizio 9 — Diagramma di flusso di un algoritmo noto

Creare il file `M1_markdown/algoritmo.md` che descrive a parole e con un diagramma Mermaid l'algoritmo di ricerca sequenziale su un array, già noto da Java. Il documento contiene un paragrafo introduttivo, un blocco di codice `java` con l'implementazione dell'algoritmo, un diagramma `flowchart` che ne rappresenta il flusso con almeno due decisioni, e un paragrafo finale che spiega che cosa succede quando l'elemento cercato non è presente.

Input di esempio: array di voti `{7, 6, 8, 5, 9}`, valore cercato `8`, indice atteso `2`.

Output atteso: su GitHub il diagramma è reso come figura, con i rombi delle decisioni e le frecce etichettate. Un errore di sintassi Mermaid produce un riquadro rosso con il numero della riga: il diagramma consegnato non deve produrlo.

Tempo stimato: 40 minuti

Esercizio 10 — Un notebook che documenta una procedura

Creare sul server il notebook `M1_esercizio10.ipynb`, che documenta passo per passo la procedura di consegna di un esercizio del corso, dalla modifica del file al `push` sul repository remoto. Il notebook alterna celle di testo e celle di codice secondo questo schema: una cella di testo iniziale con titolo, destinatario del documento e scopo; una cella di testo per ciascuno dei passi della procedura, con un titolo di secondo livello, la spiegazione in prosa del passo e il comando in un recinto `bash`; almeno una tabella che riassume i comandi e il loro effetto; una cella di testo finale con la regola di verifica prima della consegna, resa come citazione.

Le celle di codice, almeno tre, calcolano e stampano dati sulla procedura stessa: per esempio il numero di passi, i minuti stimati per l'intera procedura, il numero di file che il commit conterrà. Devono usare solo assegnamenti, aritmetica e `print`.

Vincolo di scrittura: nessuna cella di testo deve limitarsi a ripetere che cosa fa la cella di codice sottostante. Ogni cella di testo dichiara perché quel passo esiste e che cosa succede se lo si salta.

Output atteso: un notebook che un compagno può seguire per consegnare un esercizio senza chiedere nulla, e che dopo `Restart Kernel and Run All Cells` arriva in fondo senza errori con i numeri di esecuzione crescenti. Il file va copiato in `M1_jupyter/`.

Tempo stimato: 45 minuti

Esercizio 11 — Che cosa stampa un notebook eseguito fuori ordine

Un compagno consegna un notebook e dichiara che «funziona». Aprendolo si vedono quattro celle di codice nell'ordine seguente, con i numeri di esecuzione riportati fra parentesi quadre a sinistra e l'output salvato sotto ciascuna cella.

```
[2] aula = "Laboratorio 3"
    posti = 24

[4] posti = posti - 2
    print("Posti disponibili:", posti)
    Posti disponibili: 22

[3] print(aula, "-", "liberi:", posti - iscritti)
    Laboratorio 3 - liberi: 2

[1] iscritti = 22
    print("Iscritti:", iscritti)
    Iscritti: 22
```

Creare il file `M1_markdown/fuori-ordine.md` che risponde per iscritto a queste domande, ciascuna in una sezione di secondo livello. In quale ordine sono state realmente eseguite le quattro celle, e da che cosa si deduce. Perché la cella `[4]` mostra `22` mentre la cella `[3]`, eseguita prima di essa, ha calcolato con `posti` uguale a `24`. Che cosa stamperebbe ciascuna delle quattro celle dopo `Restart Kernel and Run All Cells`, riportando l'output previsto in un blocco `text` e indicando quali celle non verrebbero eseguite affatto. Quale delle quattro celle produrrebbe un errore se il notebook venisse eseguito dall'alto in basso da un kernel appena avviato, e con quale messaggio esatto. Che cosa va cambiato nel notebook perché, eseguito dall'alto in basso da un kernel vuoto, arrivi in fondo senza errori, e quale valore comparirà allora al posto di `liberi: 2`.

Output atteso: un documento di almeno cinque sezioni con le risposte motivate, che dimostra di aver distinto l'ordine di scrittura delle celle dall'ordine di esecuzione. Le previsioni di output vanno verificate ricostruendo il notebook sul server e confrontando il risultato con quanto scritto.

Tempo stimato: 40 minuti

Livello avanzato

Esercizio 12 — Architettura del progetto con due diagrammi Mermaid

Creare il file `M1_markdown/architettura.md` che documenta la struttura di un programma Java a oggetti già realizzato, con almeno tre classi. Il documento deve contenere un diagramma `classDiagram` con le classi, i loro attributi principali con l'indicazione di visibilità, i metodi pubblici e le relazioni di composizione o associazione con le molteplicità; un diagramma `sequenceDiagram` che mostra lo scambio di messaggi fra le classi durante un'operazione tipica del programma, con almeno cinque messaggi e un blocco alternativo `alt ... else`; e per ciascun diagramma un paragrafo di commento che ne spiega la lettura.

Vincolo: il diagramma delle classi non deve usare relazioni di ereditarietà, che non sono ancora state affrontate. Se il progetto scelto ne contiene, va scelto un progetto diverso oppure vanno documentate soltanto le classi indipendenti.

Output atteso: entrambi i diagrammi resi correttamente su GitHub. Il diagramma delle classi deve corrispondere al codice reale: ogni classe, ogni attributo e ogni metodo indicato deve esistere nel sorgente.

Tempo stimato: 60 minuti

Esercizio 13 — Conversione controllata con Pandoc

Prendere la relazione prodotta nell'esercizio 7 e convertirla in due formati. Il primo è un file HTML autonomo, con indice automatico limitato al secondo livello, titolo del documento impostato dai metadati e risorse incorporate, in modo che il file funzioni anche spostato su una chiavetta senza le immagini accanto. Il secondo è un file PDF impaginato con margini di 2,5 cm, corpo del carattere di 11 punti e lingua italiana.

Documentare l'operazione nel file `M1_markdown/conversione.md`, che deve contenere i due comandi esatti in blocchi `bash`, l'elenco delle opzioni usate con la spiegazione di ciascuna, e la trascrizione di eventuali messaggi di avviso ottenuti con la spiegazione di come sono stati risolti. I file `.html` e `.pdf` prodotti non vanno versionati: verificare che il `.gitignore` li escluda.

Output atteso: i due file generati si aprono correttamente, l'HTML mostra l'indice in cima con i collegamenti funzionanti, il PDF ha i margini richiesti e rende correttamente le lettere accentate.

Tempo stimato: 50 minuti

Esercizio 14 — Revisione di un README difettoso

Il docente distribuisce il file `04_Allegato_da-correggere.md`, presente nella cartella del modulo, che va salvato nel repository personale come `M1_markdown/da-correggere.md`. È un `README` che contiene difetti di sintassi Markdown e difetti di scrittura tecnica. Produrre due file. Il primo, `M1_markdown/revisione.md`, elenca i difetti trovati: per ciascuno vanno indicati la riga o la sezione, la classificazione fra difetto di sintassi e difetto di scrittura, la conseguenza per il lettore e la correzione proposta. Il secondo, `M1_markdown/corretto.md`, è la versione corretta del documento.

Non è ammesso riscrivere il documento da capo: le correzioni devono essere puntuali e il contenuto informativo va conservato, salvo quando il difetto è un'affermazione non verificabile, nel qual caso va segnalato nella revisione.

Output atteso: la revisione individua almeno otto difetti distinti, di cui almeno tre di scrittura tecnica e non di sintassi. Il documento corretto è reso senza anomalie nell'anteprima e conserva tutte le informazioni utili dell'originale.

Tempo stimato: 55 minuti

Esercizio 15 — Un notebook riproducibile e pulito prima del commit

Riprendere il notebook dell'esercizio 10 e portarlo allo stato in cui va versionato, documentando ogni passaggio nel file `M1_jupyter/riproducibilita.md`.

1. Rompere deliberatamente la riproducibilità del notebook: aggiungere una cella che definisce una variabile, usarla in una cella successiva, poi cancellare la cella che la definiva senza riavviare il kernel. Verificare che il notebook continui a funzionare a schermo e annotare perché.
2. Dimostrare il difetto con `Kernel > Restart Kernel and Run All Cells` e trascrivere nel documento il messaggio d'errore ottenuto, con il nome della cella che si ferma.
3. Correggere il notebook e ripetere la verifica fino a ottenere l'esecuzione completa.
4. Aggiungere il notebook al repository con gli output salvati, creare il `commit`, poi eseguire `jupyter nbconvert --clear-output --inplace` sul file e creare un secondo `commit`.
5. Confrontare i due `commit` con `git diff` e riportare nel documento il numero di righe cambiate in ciascuno dei due casi, con il comando usato per ottenerlo.
6. Aggiungere al `.gitignore` del repository le righe necessarie per la cartella dei salvataggi automatici di Jupyter e verificare con `git status` che la cartella non compaia più fra i file non tracciati.

Output atteso: il documento riporta il messaggio d'errore reale, i due numeri di righe cambiate a confronto e la spiegazione della differenza. Il notebook finale, riaperto su una macchina diversa o dopo un riavvio del kernel, arriva in fondo senza errori. Nella cronologia del repository i due `commit` sono distinti e i loro messaggi dichiarano che cosa contengono.

Tempo stimato: 55 minuti

Esercizio 16 — README del repository personale del corso

Riscrivere il file `README.md` nella radice del repository personale del corso, partendo dalla versione essenziale prodotta nel modulo M0 e portandola al livello che questo modulo ha reso possibile. È il prodotto finale del modulo e resterà in uso per tutto l'anno, aggiornato a ogni modulo. La versione precedente non va cancellata a mano: la sostituzione entra in un `commit` dedicato, così che `git diff` mostri che cosa è cambiato fra le due stesure.

Il documento deve contenere il titolo del repository con nome, cognome e classe; una descrizione del suo scopo in non più di quattro frasi; una tabella dei moduli con la cartella corrispondente e lo stato di avanzamento; le istruzioni per clonare il repository e per aprire i materiali,

comprese quelle per aprire i notebook sul Jupyter server d'istituto; una sezione sulle convenzioni adottate, che dichiara come si nominano le cartelle, come si scrivono i messaggi di commit, quali file non vengono versionati e come si trattano gli output dei notebook; un diagramma Mermaid a scelta che rappresenti l'organizzazione delle cartelle o il flusso di consegna di un esercizio; e una sezione finale con autore, contatto istituzionale e licenza.

Vincolo di scrittura: il documento non deve superare le settecento parole, deve usare la forma imperativa nelle istruzioni e non deve contenere alcuna affermazione non verificabile. Il grassetto è ammesso al massimo cinque volte.

Output atteso: aprendo il repository su GitHub, il `README` è reso sotto l'elenco dei file, la tabella e il diagramma sono corretti, tutti i collegamenti relativi funzionano e le informazioni corrispondono al contenuto reale del repository.

Tempo stimato: 60 minuti

Criteri di valutazione

Ogni esercizio consegnato viene valutato in decimi secondo la griglia seguente. Il peso indicato è quello usato per gli esercizi di livello intermedio e avanzato; per il livello base i criteri di scrittura tecnica e di uso di Git pesano la metà e il punteggio viene riproporzionato.

Criterio	Peso	Descrittore per la valutazione piena
Correttezza sintattica del Markdown	3	Il documento reso corrisponde all'intenzione: nessun paragrafo fuso, nessun recinto aperto, nessuna tabella o elenco non riconosciuto
Completezza rispetto alla consegna	2	Tutti gli elementi richiesti sono presenti nel numero e nella forma indicati
Struttura e gerarchia dei titoli	1	Un solo titolo di primo livello, nessun livello saltato, titoli informativi e non generici
Qualità della scrittura tecnica	2	Fraasi brevi, forma attiva, imperativo nelle istruzioni, terminologia coerente, nomi tecnici fra virgolette inverse
Verificabilità delle affermazioni	1	Comandi, versioni e percorsi corrispondono alla realtà e sono stati provati
Uso di Git	1	Il lavoro è in commit distinti, con messaggi che richiamano l'esercizio, e i file derivati non sono versionati

Gli esercizi che producono un notebook, cioè il 5, il 10, l'11 e il 15, sostituiscono i primi due criteri della griglia con quelli della tabella seguente e conservano invariati gli altri quattro.

Criterio	Peso	Descrittore per la valutazione piena
Riproducibilità	3	Dopo <code>Restart Kernel and Run All Cells</code> il notebook arriva in fondo senza errori e i numeri di esecuzione sono crescenti dall'alto verso il basso
Alternanza e qualità delle celle di testo	2	Le celle di testo usano correttamente la sintassi Markdown e dichiarano perché il codice è lì, non che cosa fa

Sono considerati errori gravi, e comportano da soli la valutazione insufficiente indipendentemente dal resto: un blocco di codice privo dell'indicazione del linguaggio; un comando riportato nel documento che non funziona se copiato e incollato; un collegamento o un'immagine con percorso assoluto riferito al proprio computer; un documento consegnato senza averne guardato l'anteprima, riconoscibile dalla presenza di costrutti resi in modo palesemente sbagliato; un notebook consegnato con numeri di esecuzione non crescenti, che dichiara da solo di non essere stato verificato con il riavvio del kernel.

Gli esercizi 12, 13, 14, 15 e 16 concorrono alla valutazione del modulo con peso doppio rispetto agli altri. Gli esercizi 15 e 16 sono obbligatori per tutti; degli esercizi 12, 13 e 14 ne va consegnato almeno uno a scelta.